

Week 4–5 Update

与上一次 week45 LaTeX PPT 相比的主要变化

2026-06-24

本次汇报只讲什么变了

- 预处理不再手写，改为直接调用 Kilosort 现成函数
- 主评估链路加入了 `float32` $\rightarrow$ `int16` 量化
- DCT 从“整段信号一个窗口”扩展为“可配置 window”
- 正确率定义进一步固定为 `baseline template matching + binned spike count comparison`
- bin 大小的选择开始和论文设置对齐
- IBL / Kilosort2.5 数据接入被正式提上日程

变化 1: 预处理与量化链路更新

相比上一次 PPT

上一次汇报里，更多是在说明 fixed-template matching 与 binned accuracy 思路；这一次已经把前处理链路改成更接近真实系统部署的版本。

- 预处理步骤改为直接调用 Kilosort 的函数，而不是自己重写同类步骤
- 重建后的 bin 不再只停留在 float32，而是补上了 int16 量化与元数据保存
- 当前主路径是：

whitened data $\rightarrow$ DCT coeffs $\rightarrow$ recon float32 $\rightarrow$ int16 transport format

- 这样做的动机是：16 bit 量化更接近真实传输约束

变化 2: keep_ratio=1.00 的基线表现已更新

Setting	Binned Micro	Binned Macro
bin = 1 s, keep_ratio = 1.00	82.37%	73.43%
bin = 500 ms, keep_ratio = 1.00	81.55%	74.46%

指标解释

$$\text{micro} = \frac{\text{total_matched_spikes}}{\text{total_baseline_spikes}}$$

$$\text{macro} = \frac{1}{N} \sum_{u=1}^N \text{accuracy}_u$$

其中 macro 是先算每个神经元的匹配率，再对神经元做平均。

变化 3: DCT 不再只支持整段信号长度

上一次 meeting 之前的逻辑

对每个通道做一次整段 DCT, 只保留前若干项:

```
for ch in range(n_chan):  
    c = dct(whitened_data[:, ch].astype(np.float32, copy=False),  
            norm='ortho')  
    coeff_mm[:, ch] = c[:n_keep].astype('float32', copy=False)
```

- 之前的隐含假设: window 就是整段信号长度
- 现在已经把 window 做成显式参数
- 这样可以模拟真实传输场景下, 数据被切成较短时间块后再压缩/发送
- 当前一个关键新配置是:

$\text{bin} = 20 \text{ ms}, \quad \text{window} = 20 \text{ ms}$

变化 4: 20 ms window / 20 ms bin 的结果已经落地

keep_ratio	Binned Micro	Binned Macro
0.10	50.74%	52.88%
0.20	75.11%	77.04%
0.30	80.92%	81.15%
0.50	81.33%	81.53%
0.70	81.32%	81.52%
1.00	81.33%	81.52%

- 这里的 20 ms 对应采样率 30 kHz 下的 600 samples
- 从结果上看, keep_ratio=0.30 之后已经接近平台区
- 和 1 s / 500 ms 相比, 短 bin 下 macro 明显更高, 说明短时局部 spike-count 结构更容易被保留

变化 5: 正确率判断逻辑已经进一步固定

- ① 从完整跑过 Kilosort 的 baseline 输出中取出:
 - templates
 - wPCA
 - ops 中 template matching 需要的参数
- ② 把这些对象作为 template_matching 的输入
- ③ 对压缩后的数据做 fixed-template matching
- ④ 再按照给定的 time bin, 比较:

$$n_{u,b}^{\text{baseline}} \quad \text{vs.} \quad n_{u,b}^{\text{compressed}}$$

- ⑤ 最终正确率衡量的是: 压缩后单个神经元在每个 bin 中的放电次数与 baseline 的差异

和上次 PPT 的区别

上次更偏“思路说明”; 这次已经把 template source、matching path、以及按 bin 统计的 evaluation path 明确固化到 notebook/workflow 里。

变化 6: bin size 的选择开始参考论文

- 查阅指定论文后, 发现 bin 往往选得很短, 主要集中在 20–80 ms
- 其中 20 ms 是最常见的选择之一
- 这支持了当前把 $\text{bin} = 20 \text{ ms}$, $\text{window} = 20 \text{ ms}$ 作为重点实验配置的做法
- 同时, 也看到在某些离线分析场景中会使用很长的 bin, 例如:
 - 800 ms
 - 更长时间尺度的热图 / 汇总分析
- 因此现在的理解是:
 - 短 bin 更适合和文献中的神经编码/传输分析对齐
 - 长 bin 更适合做稳定的离线可视化和整体统计

变化 7: baseline 定义也在继续收紧

- 除了拿原始 Kilosort baseline 作为参考，现在还加入了一个额外控制：
- 用 `keep_ratio = 1.00` 的 fixed-template rerun 结果本身作为 baseline
- 在 `window = 20 ms`, `bin = 20 ms` 下，对这个新 baseline 的比较结果是：
 - 0.30 $\rightarrow$ 97.00% / 97.00%
 - 0.50 $\rightarrow$ 99.34% / 99.35%
 - 0.70 $\rightarrow$ 99.72% / 99.74%
 - 1.00 $\rightarrow$ 100% / 100%
- 这说明原来的 82% 左右并不全是压缩误差，还混合了
 - 原始 Kilosort baseline
 - 与 fixed-template rerun 之间的方法差

变化 8: IBL 数据接入的准备情况

- internationalbrainlab.org 的数据是基于 Kilosort 2.5 的
- 因此不能直接无缝套进当前基于现有输出结构的工作流
- 预计需要补一层格式或字段转换，至少要处理：
 - output naming
 - template / spike / ops 结构差异
 - 和当前 notebook 输入约定之间的映射
- 目前硬件层面已经完成一步准备：移动硬盘已经买回来了

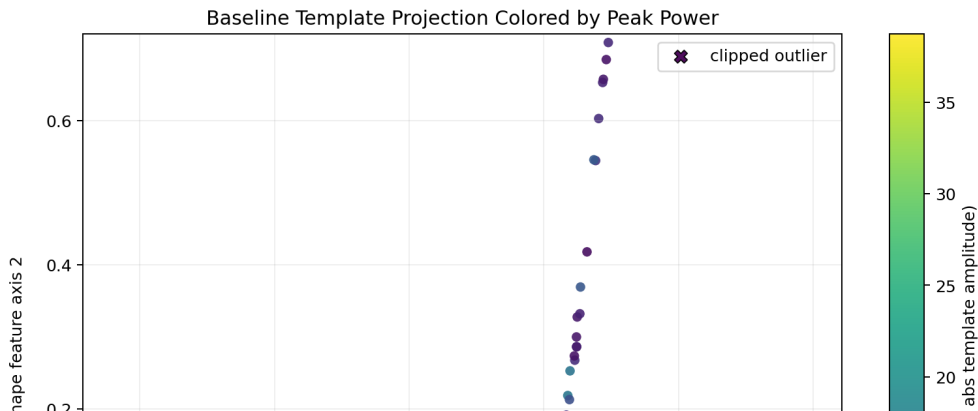
根据 IBL 官方文档，和当前项目最相关的部分

IBL 的标准组织方式是 ONE + ALF。分析时通常读取的是 `alf` 目录下的处理后结果，而不是直接从原始 `bin` 开始。

- 一个 session 下面通常有：
 - `alf/`: 处理后的行为、视频、轮子、spike sorting 等结果
 - `raw_ephys_data/probeXX/`: 原始 AP / LF 电生理数据
- spike sorting 相关结果一般在 `probe` 目录下，例如：
 - `alf/probe00/pykilosort/`
 - 或者旧版本直接在 `alf/probe00/` 下
- 最常用的对象是：
 - `spikes`
 - `clusters`
 - `channels`
- 这些对象由 IBL 的 `SpikeSortingLoader` 统一读取，而不是直接按我们当前 notebook 的 `templates.npy / eng.npy / spike_times.npy` 形式暴露

Baseline Template Projection

- 图中每个点对应一个 baseline template, 经 L2 归一化后再投影到 2D template-shape space
- 颜色表示该 template 的 peak power, 即最大绝对振幅 $\max |\text{template}|$
- 这张图已经按修正后的 notebook 逻辑重新导出, 可直接作为当前版本的诊断图



Why IBL Is Not Plug-and-Play for Current Workflow

- IBL 官方文档说明:
 - pykilosort 是默认读取版本
 - Kilosort 2.5 覆盖了大多数 2021 年 8 月之前采集的数据
- 当一个 probe 同时有多套 spike sorting 结果时:
 - 默认版本通常放在 `alf/probeXX/pykilosort`
 - 较老的 KS2.5 结果可能直接放在 `alf/probeXX/`
- 这和当前 workflow 的假设并不一致:
 - 我们现在默认本地直接有 `templates.npy`, `ops.npy`, `spike_times.npy`, `spike_clusters.npy`
 - 而 IBL 更强调通过 loader 和 collection 去解析不同 sorter / revision
- 因此接入 IBL 时至少要补一层转换:
 - collection 选择
 - spikes / clusters / channels 到当前 notebook 输入格式的映射
 - histology / channel location 元数据与本地 channel 假设的对齐

What This Means for the Next Step

- ① 先用 IBL loader 稳定读出某个 probe 的 spikes, clusters, channels
- ② 明确该 probe 使用的是:
 - pykilosort
 - 还是 Kilosort 2.5
- ③ 再把这些对象转换成当前固定模板匹配链路所需的 baseline 输入
- ④ 最后才把压缩、量化、template matching、binned accuracy 这一整条链路套进去

本页信息来源

IBL 官方文档:

- ONE Quick Start
- Get to know the datasets and folder structure
- Loading SpikeSorting Data

- 这次相对上一次 week45 PPT, 最大的变化不是“新模型”, 而是 **workflows 更接近真实系统实现**
- 预处理改成直接调 Kilosort, 量化补到 int16, DCT 改成支持 window
- $\text{bin} / \text{window} = 20 \text{ ms}$ 的实验已经跑通, 并且结果开始和论文常见设置对齐
- 准确率定义现在更清楚地区分了:
 - 相对原始 Kilosort baseline 的误差
 - 相对 fixed-template rerun baseline 的纯压缩误差
- 下一步重点会转向:
 - 更系统地扫描短 bin / 短 window 配置
 - IBL / Kilosort2.5 数据适配